Hierarchical LLM-Based Planning and Replanning for Autonomous Multi-Drone Systems

Ákos Varga

Master's Thesis in Computer Engineering

Department of Electrical and Computer Engineering

Aarhus University

Project period: Spring 2026

Supervisor: Andriy Sarabakha

Abstract

Preface

Contents

Abstract i

Preface ii

List of Figures v

List of Tables vi

Abbreviations vii

1 Introduction 1

1.1 Problem Statement 1

1.2 Research Goals 2

1.3 Thesis Structure 2

2 Background 3

2.1 Autonomous Drone Systems 3

2.2 Multi-Agent Systems 4

2.3 Multi-Agent Systems 4

2.3.1 Coordination in Multi-Agent Systems 5

2.3.2 Planning in Multi-Agent Systems 5

2.4 Large Language Models 7

2.4.1 Fundamentals of Large Language Models 7

2.4.2 Transformer Architecture 7

2.4.3 Model Scaling and Deployment Considerations 7

2.4.4 LLMs in Multi-Agent Systems 8

2.5 Related Work 8

3 Methodology **11**

3.1 System Overview 11

3.2 Task Decomposition 13

3.2.1 Example 13

3.3 Task Allocation 14

3.3.1 Example 15

3.4 Task Scheduling 16

3.4.1 Example 17

3.5 Replanning 18

3.6 System Architecture 20

3.6.1 Overview 20

3.6.2 System Components 21

4 Results **24**

4.1 Component-Level LLM Evaluation 24

4.1.1 Cloud-based Planning Pipeline 24

4.1.2 Onboard Task Admission 26

4.2 Simulation Results 27

4.2.1 Simulation Setup 28

4.2.2 Simulation Case Study 28

4.3 Physical Platform and Experimental Setup 33

4.3.1 Scope of Physical Integration 33

4.3.2 Drone Platform 33

4.3.3 Laboratory and Control Setup 34

5 Conclusion and Future Work **36**

List of Figures

3.1 Caption for the horizontal graph. 12

3.2 Software architecture of the implemented multi-drone planning and replanning system. The planner process receives the world configuration, calls the planning pipeline, dispatches commands through per-drone command queues, and receives execution events through a shared event queue. 21

4.1 Initial stages of the simulated mission execution. 30

4.2 Intermediate stages of the simulated mission execution. 31

4.3 Final stages of the simulated mission execution. 32

4.4 Drone platforms used during physical testing. 33

List of Tables

2.1 Comparison of LLM-based multi-agent systems. 10

4.1 Average makespan difference from the VRP baseline by number of subtasks. Values in parentheses indicate solved cases over three tasks. 25

4.2 Average inference time by number of subtasks. Values in parentheses indicate solved cases over three tasks. 26

4.3 Task admission accuracy across failure categories. Each value indicates the number of correctly solved cases out of 20. 27

4.4 Average and maximum inference time for the task admission module. 27

4.5 Capabilities of the Anafi drone variants used in the project. 34

Abbreviations

CPU	Central Processing Unit
GPU	Graphics Processing Unit
LLM	Large Language Model
UAV	Unmanned Aerial Vehicle
VRP	Vehicle Routing Problem

Chapter 1

Introduction

1.1 Problem Statement

Autonomous drone systems are increasingly deployed in applications such as infrastructure inspection [1], surveillance [2], and environmental monitoring [3]. While reliable low-level flight control and sensing capabilities have been widely developed, expressing high-level mission objectives in a way that is both intuitive for human operators and executable by autonomous systems remains a challenge.

Existing approaches rely on predefined workflows or formal planning models, which require expert knowledge and lacks flexibility when faced with dynamic, changing environments. As a result, translating human instructions into concrete mission plans remains a bottleneck.

Recent advances in Large Language Models (LLMs) offer a promising approach for bridging this gap. LLMs can interpret natural language instructions and infer task structure, enabling them to transform human mission descriptions into structured plans that autonomous systems can execute. This capability makes them well suited for high-level reasoning tasks such as task decomposition, allocation, and scheduling in multi-agent systems. This project proposes the use of LLMs to generate executable plans for multi-drone missions. Given a natural language mission description, the system decomposes the task into atomic subtasks, allocates them to drones according to their capabilities, and generates a structured execution schedule.

To address the challenges of real-world deployment, the system adopts a hierarchical architecture in which cloud-based LLMs perform high-level mission planning,while smaller locally deployed models monitor system state and enable adaptive replanning in response to failures, delays, or environmental changes. The goal is to develop a flexible and robust framework capable of translating human intent into coordinated multi-drone behavior under realistic constraints.

1.2 Research Goals

The primary goal of this thesis is to investigate the feasibility and effectiveness of using Large Language Models for high-level planning and adaptive replanning in multi-drone systems. While LLMs have demonstrated strong capabilities in natural language understanding and reasoning, their applicability to real-time, safety-critical autonomous systems remains an open question.

This thesis aims to explore the following research goals:

- To evaluate how effectively LLMs can translate natural language mission descriptions into structured, executable task plans for multi-drone systems.
- To investigate the ability of LLMs to perform task decomposition, allocation, and scheduling in environments with heterogeneous agents and constrained resources.
- To assess the robustness of LLM-based planning when operating in dynamic environments that require replanning due to failures, delays, or environmental changes.
- To analyze the trade-offs between cloud-based and locally deployed models in terms of latency, reliability, and responsiveness during mission execution.
- To identify the limitations of LLM-based planning approaches and determine under which conditions they outperform or fall short of traditional planning methods.

1.3 Thesis Structure

This thesis is organized as follows:

Chapter 2 reviews the technical background of the project and explores related work on autonomous multi agent systems using Large Language Models for reasoning and decision-making.

Chapter 3 presents the overall system architecture, including the design of the LLM-based planning pipeline and the hierarchical replanning mechanism as well as integration with drone control interfaces.

Chapter 4 presents the experimental setup and evaluation methodology, including simulation and real-world test scenarios, as well as performance metrics.

Chapter 5 discusses the results, analyzing the effectiveness, robustness, and limitations of the proposed approach.

Chapter 6 concludes the thesis and outlines directions for future work.

Chapter 2

Background

2.1 Autonomous Drone Systems

Autonomous drone systems are becoming increasingly prominent in applications that are otherwise time-consuming or dangerous using traditional methods. Such applications include infrastructure inspection, for example monitoring bridge structures [4], environmental monitoring such as wildlife surveys [3], and search and rescue (SAR) missions, where drones can assist in locating people after disasters [5]. Unmanned Aerial Vehicles (UAVs) are also widely used in surveillance and security applications, including border monitoring and crowd observation [6]. In addition, drones have been successfully deployed for logistics and delivery tasks, such as Amazon Prime Air [7], as well as real-world medical transport solutions, including long-range delivery of blood samples in Denmark [8]. Large-scale industrial and energy infrastructures can likewise be efficiently monitored using UAVs [9].

These diverse applications highlight the potential of UAVs; however, understanding the characteristics and capabilities of different drone platforms is essential.

Multirotor drones are widely used due to their ability to hover, perform vertical takeoff and landing (VTOL), and navigate precisely in constrained environments. These capabilities make them particularly suitable for urban operations such as inspection and delivery. In contrast, fixed-wing drones are designed for long-endurance missions and can cover large areas efficiently, making them well suited for large-scale mapping and environmental monitoring tasks. However, they lack hovering capability and require more space for takeoff and landing, limiting their use in high-precision or urban scenarios.

The suitability of a drone for a given mission depends heavily on the payload it carries. RGB cameras provide visual data for inspection, surveillance, and mapping tasks, and can also be used for motion estimation through visual odometry [10]. Thermal cameras mounted on UAVs have been successfully applied in infrastructure inspection, for example to detect subsurface delamination in bridge decks using infrared thermography [4]. LiDAR-equipped UAVs enable accurate three-dimensional mapping of environments, for instance in forestry applications

where canopy structure can be estimated from aerial point clouds [11]. In addition, onboard sensors such as Inertial Measurement Units (IMUs), GPS, and barometers are used for state estimation, including position, velocity, and altitude.

In multi-drone systems, payload diversity introduces functional heterogeneity, meaning that not all drones are capable of performing all tasks. This has direct implications for task allocation, as tasks must be matched with drones that possess the required sensing and operational capabilities.

Many modern UAV applications rely on artificial intelligence (AI) for tasks such as perception, navigation, and decision-making. However, developers often face strict hardware constraints in drone systems. While cloud-based AI can provide significant computational resources, it introduces latency and dependence on network availability. The emergence of edge AI enables real-time decision-making by allowing drones to process sensor data locally and operate independently of external infrastructure [12]. These platforms often support hardware acceleration through GPUs, DSPs, or NPUs, enabling efficient execution of machine learning models. This is particularly relevant for deploying compact and quantized large language models (LLMs) directly onboard drones.

2.2 Multi-Agent Systems

Multi-agent systems (MAS) consist of multiple autonomous agents that interact in a shared environment to achieve a collective goal [13]. In the context of autonomous drone systems each drone can be modelled as an autonomous agent with it's own sensing and decision making capabilities. Deploying heterogeneous UAV agents increases the number of available data modalities, the area that the drone swarm can inspect and provides backup options in case of drone failures, thereby increasing mission success rate in constrast to a single agent solution [14]. The key challenges in multi-agent systems are the efficient coordination between agents and the creation of a mission plan for heterogeneous agents, particularly under constraints such as limited communication, energy, and computation.

2.3 Multi-Agent Systems

Multi-agent systems (MAS) consist of multiple autonomous agents that interact within a shared environment to achieve individual or collective goals. In the context of autonomous drone systems, each drone can be modeled as an autonomous agent with its own sensing, computation, communication, and decision-making capabilities. Deploying heterogeneous UAV agents can increase the number of available sensing modalities, expand the area that can be inspected, and provide redundancy in case of drone failures, thereby improving mission robustness compared to a single-agent solution [14]. However, multi-agent systems also introduce challenges related to coordination, task allocation, and mission planning,

particularly under constraints such as limited communication bandwidth, finite battery capacity, and restricted onboard computation.

2.3.1 Coordination in Multi-Agent Systems

Two common architectural approaches for coordination in multi-agent systems are centralized and decentralized coordination. In a centralized architecture, a base station or ground control station is responsible for critical mission decisions and coordinates the behavior of the agents. Since the central unit is often equipped with greater computational resources than the individual agents, centralized approaches can support more complex planning and optimization. However, this architecture introduces a single point of failure: if the base station fails or communication with it is lost, the mission may be compromised.

In a decentralized architecture, each agent is capable of local decision-making and may communicate directly with other agents. This can improve robustness, since the system can remain operational even if individual agents fail. Decentralized systems may also scale better as new agents are added, since control is not concentrated in a single central unit. However, coordination becomes more challenging because agents must reach decisions using local information and distributed communication mechanisms.

Several mechanisms can support decentralized coordination. Leader election methods dynamically select a coordinating agent responsible for organizing group decisions. Auction-based methods allocate tasks by allowing agents to bid based on local cost, capability, or availability, with tasks assigned according to the most suitable bid. Voting mechanisms allow agents to collectively choose between alternatives, while bargaining and negotiation approaches enable agents to iteratively exchange proposals until an agreement is reached.

Hybrid approaches combine elements of both centralized and decentralized coordination. For example, a ground station may perform high-level mission planning, while individual UAVs retain autonomy to make local decisions based on their current state and surrounding environment. Such architectures can exploit the computational resources of centralized systems while preserving some of the robustness and adaptability of decentralized systems.

2.3.2 Planning in Multi-Agent Systems

In multi-agent systems with heterogeneous agents, the task allocation problem consists of assigning tasks to agents under given constraints, such as energy, capability, and communication limitations, while optimizing a global objective, for example minimizing mission time or maximizing coverage. Several approaches to multi-agent planning have been identified in the literature [15].

One approach is centralized planning for distributed execution, where a central planner constructs a global plan for all agents. This plan specifies both the allocation of tasks and the ordering of actions, and is subsequently distributed to individual agents for execution.

While this approach simplifies coordination by leveraging a global view of the system, it relies on a central entity and is therefore susceptible to single points of failure.

An alternative approach is distributed planning with centralized plan generation, where multiple agents collaborate to construct a single global plan. In this case, agents may contribute specialized knowledge or capabilities during the planning phase, but the resulting plan is still shared among agents for execution. This approach can improve flexibility in plan generation, although it still assumes the existence of a coherent global plan.

The most decentralized approach is distributed planning for distributed execution, where each agent independently generates its own plan while coordinating with others during execution. In such systems, agents may only possess partial knowledge of the overall mission and must dynamically resolve conflicts and dependencies. Coordination in this setting often relies on negotiation, communication, or local decision-making mechanisms. As a result, a fully global plan may never exist, and system behavior emerges from the interaction of individual agent plans.

In general, centralized planning approaches are simpler to design and can produce globally consistent plans due to their access to complete system information. However, decentralized approaches offer greater robustness and scalability, as they avoid reliance on a single coordinating entity. The choice between these approaches depends on the requirements of the system, including the need for global optimality, robustness to failures, and adaptability to dynamic environments.

Many multi-agent planning problems can be formulated as optimization problems. A common abstraction is the multi-traveling salesman problem (mTSP), where multiple agents must visit a set of locations while minimizing a cost function. In this formulation, tasks correspond to nodes, agents correspond to salesmen, and decision variables determine the assignment and ordering of tasks for each agent. While the multi-traveling salesman problem (mTSP) provides a useful abstraction, more realistic formulations are given by the Vehicle Routing Problem (VRP). In the VRP, the objective is to determine optimal routes for multiple vehicles that must visit a set of locations. In the special case of a single vehicle, the problem reduces to the classical Traveling Salesperson Problem (TSP). The notion of optimality in VRP depends on the application context. A common objective is to minimize the total distance traveled by all vehicles; however, this may lead to unbalanced solutions where a single agent performs most tasks. An alternative objective is to minimize the length of the longest route, which promotes balanced workload distribution and reduces overall mission completion time. Furthermore, VRP formulations can incorporate additional constraints, such as capacity limits and time windows, which allow modeling of practical considerations like energy constraints and task deadlines in multi-drone systems.

Different solution approaches exist for such problems. Exact methods, such as Mixed Integer Linear Programming (MILP), can provide optimal solutions but often suffer from high computational complexity and limited scalability. In contrast, heuristic and metaheuristic

methods, such as genetic algorithms, can provide faster solutions and are more suitable for large-scale problems, albeit without guarantees of optimality [16].

Finally, real-world multi-agent systems must handle dynamic changes, such as agent failures. In such cases, replanning is required, which can be performed either centrally by a global planner or locally by individual agents. Hybrid approaches may combine both strategies, allowing systems to balance global optimality with robustness and adaptability.

2.4 Large Language Models

2.4.1 Fundamentals of Large Language Models

Large Language Models (LLMs) are computational systems designed to generate and understand natural language by modeling the statistical structure of text. At their core, LLMs estimate the probability distribution of the next token given a sequence of preceding tokens. Formally, given a context of $\omega_1, \omega_2, \ldots, \omega_{t-1}$, the model predicts the next token w_t, thereby enabling both language understanding and generation. The key idea that enables modern LLMs is pretraining, where models learn linguistic structure and world knowledge by repeatedly predicting the next token over very large text corpora.

2.4.2 Transformer Architecture

Modern LLMs are implemented using the Transformer architecture introduced in Attention is All You Need [17]. The core component of the Transformer is the self-attention mechanism, which enables the model to weigh the importance of different tokens relative to each other when generating representations. Given an input sequence, attention computes interactions between all token pairs, allowing the model to capture long-range dependencies efficiently.

2.4.3 Model Scaling and Deployment Considerations

Large Language Models vary significantly in size, ranging from millions to hundreds of billions of parameters. While larger models generally achieve higher performance due to increased representational capacity, they also require substantial computational resources, including high memory bandwidth and specialized hardware such as GPUs. This poses a challenge for deployment in resource-constrained environments, such as embedded systems, mobile devices, or robotic platforms.

A common approach to address these limitations is quantization, where model weights are represented with reduced numerical precision (e.g., 8-bit or 4-bit instead of 32-bit floating point). Quantized LLMs significantly reduce memory usage and computational requirements, enabling deployment on edge devices while maintaining acceptable performance.

2.4.4 LLMs in Multi-Agent Systems

Large Language Models (LLMs) have been successfully applied in various domains, including multi-agent systems, due to their strong reasoning abilities, contextual awareness, and generalization capabilities. In this context, their applications can be broadly categorized into communication, perception, planning, and control, following [18].

LLMs are particularly effective at handling the ambiguity of natural language and can be used for both *interpretation* and *grounding*. Interpretation refers to converting natural language into structured representations, such as code or formal planning languages (e.g., PDDL). Grounding involves mapping human instructions to objects, actions, or processes that can be recognized and executed by agents.

Perception enables agents to operate in real-world environments. Language models have been extended to multimodal settings by integrating visual and auditory inputs. These models, such as Vision-Language Models (VLMs) and Audio Language Models (ALMs), process non-textual data alongside language, allowing agents to detect objects and interpret signals from their surroundings.

In planning, traditional approaches rely on predefined algorithms that must be adapted whenever the environment or objectives change. In contrast, LLM-based planning through prompting allows for flexible task specification and the incorporation of environmental feedback. However, due to the risk of hallucinations, LLM-generated plans should be validated against system constraints or combined with external planning methods.

For control, LLMs can be used in two main ways. In the direct approach, the model maps natural language instructions directly to control commands or actions. In the indirect approach, the model generates higher-level outputs such as subgoals or plans, which are then executed by separate control systems. The indirect approach is generally more flexible and better suited for complex and multi-agent scenarios, although it still requires reliable grounding of language into physical actions.

2.5 Related Work

Among early LLM-based planners for automous agents, SayCan [19] introduces a framework that grounds language model reasoning in real-world robot capabilities. In this approach, an LLM generates candidate low-level skills from a high-level natural language instruction. Each skill is evaluated using two complementary scores: (i) a language model score, which measures how relevant the skill is for completing the task, and (ii) an affordance score, which estimates the feasibility of executing the skill given the robot's current state and environment. These scores are combined to select the most appropriate action, and the process is iteratively repeated until the task is completed. This work demonstrates that LLMs can be effectively applied to robotics when their outputs are constrained by physical feasibility.

In the context of UAVs, Wang et al. [20] introduce the GSCE framework, which combines guidelines, skill APIs, constraints within a single prompt. By leveraging step-by-step Chain-of-Thought (CoT) reasoning together with few-shot examples, the system generates executable code for a single drone. While SayCan and GSCE focus on single-agent systems, they highlight the potential of LLMs for grounded reasoning and executable plan generation.

Extending these ideas to multi-agent systems, RoCo [21] introduces a collaborative reasoning framework for multi-robot coordination. In RoCo, each robot is associated with an LLM-based agent that has access to its own capabilities, local observations, and partial knowledge of the environment. These agents communicate through a structured dialogue process, iteratively proposing, refining, and critiquing task plans. Through this interaction, the system converges to a shared global plan, which is then decomposed and distributed among the robots for execution. Additionally, RoCo leverages LLMs during execution by generating 3D waypoints, enabling more flexible behavior.

Building on multi-agent coordination, SMART-LLM [22] introduces a structured framework for task planning in multi-agent systems, generating executable mission plans through a three-stage pipeline, each implemented by a dedicated LLM agent. In the first stage, *task decomposition*, the LLM converts a high-level natural language instruction into a set of independent subtasks. Each subtask is represented as a sequence of actions, while incorporating contextual information about the environment. The second stage, *coalition formation*, considers the available robots and their capabilities, and assigns groups of robots to each subtask such that every coalition collectively satisfies the required skill set for execution. In the final stage, *task allocation*, the system generates executable code for each robot based on the assigned subtasks and coalitions, resulting in a distributed execution plan. This work highlights the importance of decomposing complex planning problems into modular steps, where each stage focuses on a specific aspect of the planning process, and demonstrates the effectiveness of few-shot prompting in improving the quality and consistency of LLM outputs.

To enable adaptive behavior in dynamic environments, CoMuRoS [23] is another framework that incorporates a hierarchical architecture with event-driven replanning. The system employs a central LLM (Task Manager) for high-level reasoning, including interpreting natural language goals, decomposing tasks, allocating subtasks, and updating plans based on environmental feedback. Each robot is equipped with a local LLM that translates assigned tasks into executable Python code and performs event classification, distinguishing between relevant and irrelevant observations. Relevant events are communicated back to the Task Manager to trigger replanning. While the system demonstrates strong adaptability and has been validated in real-world scenarios with heterogeneous robots, including drones and quadrupeds, it primarily focuses on feasibility and reactive coordination. In particular, the generated plans lack explicit temporal reasoning, as tasks are not associated with execution times or synchronization constraints between agents. Furthermore, the framework does not incorporate a clear objective function, such as minimizing total mission time, leading to potentially suboptimal utilization of resources.

Table 2.1: Comparison of LLM-based multi-agent systems.

	SayCan	GSCE	RoCo	SMART-LLM	CoMuRoS	AutoHMA	Ours
Year	2022	2025	2023	2024	2025	2025	2026
Multi-Robot	✗	✗	✓	✓	✓	✓	✓
Heterogeneous	✓	✗	✓	✓	✓	✓	✓
Framework	Centralized	Centralized	Decentralized	Centralized	Hybrid	Hybrid	Centralized
Replanning	✗	✗	✓	✗	✓	✓	✓
Hardware Demo	✓	✗	✓	✗	✓	✗	✓
Scheduling	✗	✗	✗	✗	✗	✓	✓
Objective Function	✗	✗	✗	✗	✗	✓	✓

Complementary to replanning-focused approaches, AutoHMA-LLM [24] proposes a hierarchical architecture that combines cloud-based LLMs, device-level LLMs, and classical control algorithms to address task coordination and scheduling. In this framework, the cloud LLM performs global task decomposition, allocation, and scheduling, while device-level LLMs translate tasks into robot-specific instructions executed by classical control methods. The system explores multiple coordination strategies and demonstrates that a centralized hybrid model achieves the best trade-off between efficiency and performance through continuous feedback between system layers. Although the framework formulates scheduling as an optimization problem, it does not produce explicit, time-parameterized schedules that can be directly inspected or executed. Additionally, the system is evaluated primarily in simulation environments, without validation on real robotic platforms, leaving its performance under real-world conditions unverified.

Taken together, these works demonstrate the growing capabilities of LLM-based multi-agent systems in task decomposition, coordination, and adaptive replanning. However, they also reveal a key limitation: the absence of explicit scheduling mechanisms that incorporate temporal constraints and objective-driven optimization. As shown in Table 2.1, while several approaches support multi-agent coordination and, in some cases, dynamic replanning, none of them provide explicit, time-parameterized schedule that enable efficient and coordinated multi-agent operation.

Chapter 3

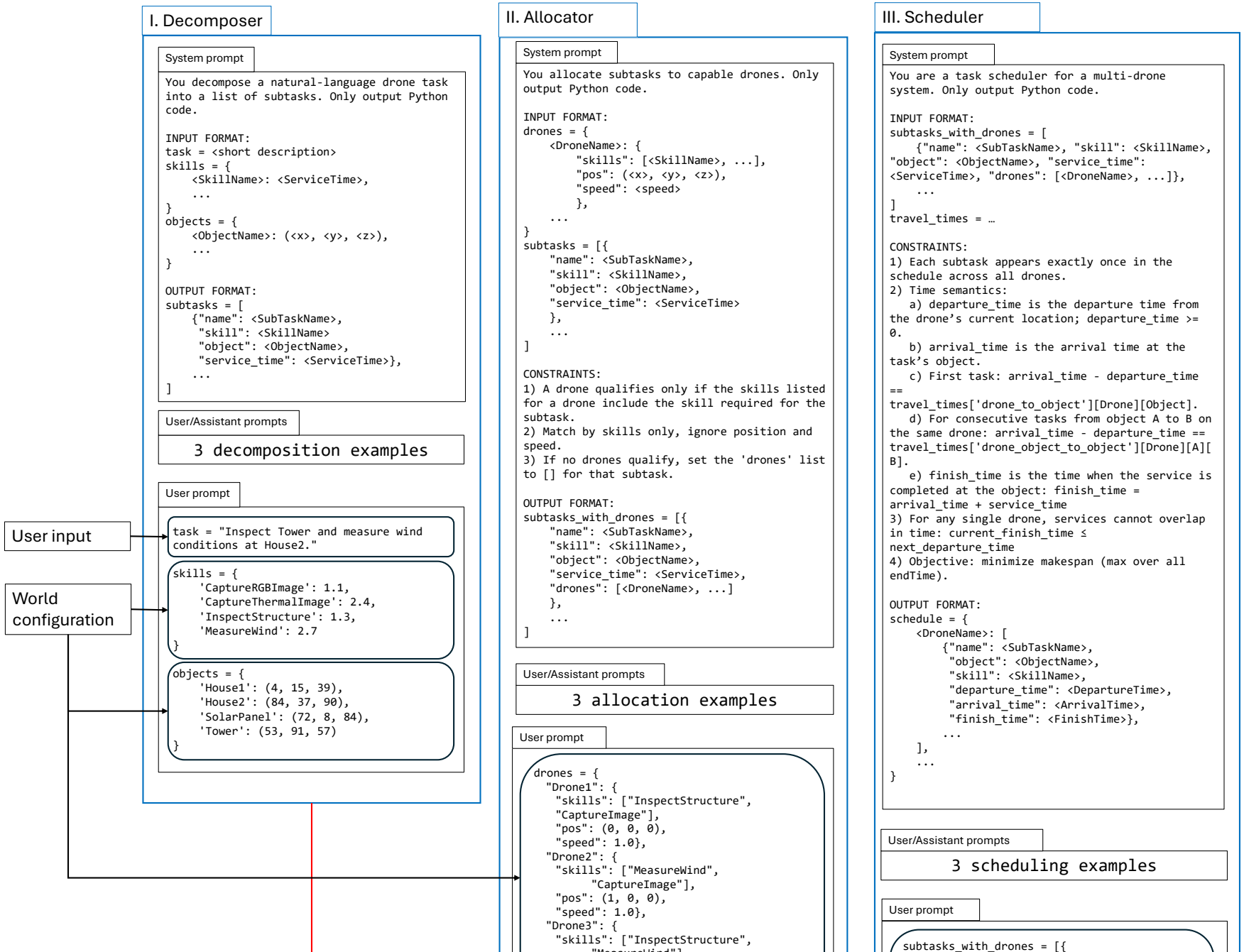
Methodology

3.1 System Overview

The core of the system is a cloud-based multi-drone task scheduling pipeline. Given a high-level mission command from a user, the system creates a time-parameterized schedule for a team of heterogeneous drones. The pipeline consists of three stages: task decomposition, task allocation, and task scheduling. First, the task decomposition stage converts the natural-language mission into a list of atomic subtasks. Second, the task allocation stage assigns each subtask a set of suitable candidate drones based on the required skill for the subtask and the capabilities of each drone. Finally, the scheduling stage selects the final drone for each subtask and determines the temporal order of execution, taking into account travel time and task execution duration, while aiming to minimize the overall mission makespan.

The output of the pipeline is a per-drone schedule with departure, arrival, and finish times for each subtask. This representation enables coordinated multi-drone operation and allows the generated plan to be inspected before flight. An overview of the pipeline with an example task is shown in Figure 3.1.

The system also supports replanning in case of drone or task failures, such as critically low battery level, poor connection, or a crash. In such cases, the planner updates the set of available drones and regenerates the schedule for unfinished or interrupted subtasks.


I. Decomposer
System prompt
You decompose a natural-language drone task into a list of subtasks. Only output Python code.
INPUT FORMAT:
task = <short description>
skills = {
<SkillName>: <ServiceTime>,
...
}
objects = {
<ObjectName>: (<x>, <y>, <z>),
...
}
OUTPUT FORMAT:
subtasks = [
{"name": <SubTaskName>,
"skill": <SkillName>
"object": <ObjectName>,
"service_time": <ServiceTime>},
...
]
User/Assistant prompts
3 decomposition examples
User prompt
User input
task = "Inspect Tower and measure wind conditions at House2."
World configuration
skills = {
'CaptureRGBImage': 1.1,
'CaptureThermalImage': 2.4,
'InspectStructure': 1.3,
'MeasureWind': 2.7
}
objects = {
'House1': (4, 15, 39),
'House2': (84, 37, 90),
'SolarPanel': (72, 8, 84),
'Tower': (53, 91, 57)
}
II. Allocator
System prompt
You allocate subtasks to capable drones. Only output Python code.
INPUT FORMAT:
drones = {
<DroneName>: {
"skills": [<SkillName>, ...],
"pos": (<x>, <y>, <z>),
"speed": <speed>
},
...
}
subtasks = [{
"name": <SubTaskName>,
"skill": <SkillName>,
"object": <ObjectName>,
"service_time": <ServiceTime>
},
...
]
CONSTRAINTS:
1) A drone qualifies only if the skills listed for a drone include the skill required for the subtask.
2) Match by skills only, ignore position and speed.
3) If no drones qualify, set the 'drones' list to [] for that subtask.
OUTPUT FORMAT:
subtasks_with_drones = [{
"name": <SubTaskName>,
"skill": <SkillName>,
"object": <ObjectName>,
"service_time": <ServiceTime>,
"drones": [<DroneName>, ...]
},
...
]
User/Assistant prompts
3 allocation examples
User prompt
drones = {
"Drone1": {
"skills": ["InspectStructure",
"CaptureImage"],
"pos": (0, 0, 0),
"speed": 1.0},
"Drone2": {
"skills": ["MeasureWind",
"CaptureImage"],
"pos": (1, 0, 0),
"speed": 1.0},
"Drone3": {
"skills": ["InspectStructure",
III. Scheduler
System prompt
You are a task scheduler for a multi-drone system. Only output Python code.
INPUT FORMAT:
subtasks_with_drones = [
{"name": <SubTaskName>, "skill": <SkillName>, "object": <ObjectName>, "service_time": <ServiceTime>, "drones": [<DroneName>, ...]},
...
]
travel_times = …
CONSTRAINTS:
1) Each subtask appears exactly once in the schedule across all drones.
2) Time semantics:
a) departure_time is the departure time from the drone's current location; departure_time >= 0.
b) arrival_time is the arrival time at the task's object.
c) First task: arrival_time - departure_time == travel_times['drone_to_object'][Drone][Object].
d) For consecutive tasks from object A to B on the same drone: arrival_time - departure_time == travel_times['drone_object_to_object'][Drone][A][B].
e) finish_time is the time when the service is completed at the object: finish_time = arrival_time + service_time
3) For any single drone, services cannot overlap in time: current_finish_time ≤ next_departure_time
4) Objective: minimize makespan (max over all endTime).
OUTPUT FORMAT:
schedule = {
<DroneName>: [
{"name": <SubTaskName>,
"object": <ObjectName>,
"skill": <SkillName>,
"departure_time": <DepartureTime>,
"arrival_time": <ArrivalTime>,
"finish_time": <FinishTime>},
...
],
...
}
User/Assistant prompts
3 scheduling examples
User prompt
subtasks_with_drones = [{

3.2 Task Decomposition

The task decomposition phase transforms high-level mission commands into a structured set of subtasks. A mission instruction may contain several objectives, such as inspecting different structures, capturing images, recording videos, or collecting environmental measurements. The atomic subtasks generated by the decomposer can later be assigned to drones and scheduled independently.

The decomposition stage is performed using an LLM prompted with the available environmental objects, supported drone skills, their estimated execution times, the required output format, and few-shot examples. The model is instructed to generate only subtasks that can be expressed using the predefined skill set. This constrains the LLM output and reduces the risk of hallucinated actions that cannot be executed by the system.

The output of this stage can be represented as:

$$S = \{s_1, s_2, \ldots, s_n\},$$

where each subtask s_i is defined as:

$$s_i = (name_i, skill_i, object_i, service_i).$$

Here, $name_i$ is the subtask identifier, $skill_i$ is the required drone capability, $object_i$ is the target object in the environment, and $service_i$ is the estimated time required to execute the skill at the target object.

3.2.1 Example

Given the user-defined mission instruction `"Inspect Tower and measure wind conditions at House2."`, together with the available skills and objects defined in the world configuration:

```
skills = {
    'CaptureRGBImage': 1.1,
    'CaptureThermalImage': 2.4,
    'InspectStructure': 1.3,
    'MeasureWind': 2.7,
}

objects = {
    'House1': (4, 15, 39),
    'House2': (84, 37, 90),
    'SolarPanel': (72, 8, 84),
    'Tower': (53, 91, 57),
}
```

the decomposer generates the following atomic subtasks:

```
subtasks = [
    {
        "name": "SubTask1",
        "skill": "InspectStructure",
        "object": "Tower",
        "service_time": 2.0,
    },
    {
        "name": "SubTask2",
        "skill": "MeasureWind",
        "object": "House2",
        "service_time": 1.5,
    },
]
```

3.3 Task Allocation

The task allocator identifies which drones are capable of executing each decomposed subtask, creating a list of candidate drones whose skill set includes the skill required by the subtask. The allocator determines which assignments are possible, while the scheduler later selects the final drone for each subtask based on temporal criteria such as flight time and availability.

Each drone is described by its available skills, current position, and speed. Each subtask is described by its name, required skill, target object, and estimated service time. In the allocation stage, only skill information is used. A drone qualifies for a subtask if the required skill of the subtask is included in the drone's skill list. Position and speed are intentionally ignored at this stage, since they are considered later by the scheduler when evaluating timing and efficiency.

Formally, let $D = \{d_1, d_2, \ldots, d_m\}$ denote the set of drones, and let K_j be the set of skills available to drone d_j. For a subtask s_i requiring skill $skill_i$, the candidate drone set is defined as:

$$C_i = \{d_j \in D \mid skill_i \in K_j\}.$$

If no drone satisfies the required skill, then $C_i = \emptyset$, indicating that the subtask is infeasible with the current drone team.

The allocator is implemented using few-shot prompting. The prompt provides the LLM with the required input format, allocation constraints, and examples of valid allocations. The model is instructed to output only structured Python code in the form of a list of dictionaries. Each dictionary preserves the original subtask information and adds a list of candidate drones capable of performing the task.

3.3.1 Example

Using the decomposed subtasks from Section 3.2.1 and the following drone configuration:

```
drones = {
    "Drone1":
    {
        "skills": ["InspectStructure", "CaptureImage"],
        "pos": (0, 0, 0),
        "speed": 1.0
    },
    "Drone2":
    {
        "skills": ["MeasureWind", "CaptureImage"],
        "pos": (1, 0, 0),
        "speed": 1.0
    },
    "Drone3":
    {
        "skills": ["InspectStructure", "MeasureWind"],
        "pos": (0, 1, 0),
        "speed": 1.0
    }
}
```

`SubTask1` requires the `InspectStructure` skill. Therefore, `Drone1` and `Drone3` are included in its candidate drone set. Similarly, `SubTask2` requires the `MeasureWind` skill, which can be performed by `Drone2` and `Drone3`. At this stage, the allocator does not select the final executing drone; it only identifies the feasible candidates for each subtask. The resulting allocated subtasks are:

```
subtasks_with_drones = [
    {
        "name": "SubTask1",
        "skill": "InspectStructure",
        "object": "Tower",
        "service_time": 2.0,
        "drones": ["Drone1", "Drone3"]
    },
    {
        "name": "SubTask2",
        "skill": "MeasureWind",
        "object": "House2",
        "service_time": 1.5,
        "drones": ["Drone2", "Drone3"]
    }
]
```

3.4 Task Scheduling

The scheduler converts the allocated subtasks into a time-parameterized multi-drone execution schedule. While task decomposition determines what must be done and task allocation determines which drones can perform each task, scheduling assigns the final drone to each subtask and determines when each task should be executed.

The scheduler receives two inputs: the list of subtasks with their candidate drones from the task allocator, and a travel-time table describing the estimated travel time from each drone's initial position to each object, as well as between pairs of objects. The travel-time table is precomputed before scheduling in order to reduce the arithmetic burden on the LLM and ensure consistent timing estimates. This allows the LLM to focus on the combinatorial scheduling decision, namely selecting drones, ordering tasks, and minimizing the overall makespan, rather than repeatedly computing distances and travel times during generation.

The scheduler is constrained to assign each subtask exactly once across the drone team. Furthermore, a subtask may only be assigned to a drone contained in its candidate drone list. For each scheduled task, three temporal attributes are produced: *departure_time*, *arrival_time*, and *finish_time*. The departure time denotes when a drone starts travelling toward the target object, the arrival time denotes when it reaches the object, and the finish time denotes when the required service has been completed. For a first task assigned to a drone, the arrival time is computed using the travel time from the drone's initial position to the task object. For consecutive tasks assigned to the same drone, the arrival time is computed using the travel time from the previous task object to the next task object. The finish time is then given by the arrival time plus the service time of the task.

Formally, for a task s_i assigned to drone d_j, the scheduled task is represented as:

$$\tau_i = (s_i, d_j, t_i^{dep}, t_i^{arr}, t_i^{fin}).$$

If s_i is the first task assigned to drone d_j, then:

$$t_i^{arr} - t_i^{dep} = T_{j,o_i}^{start},$$

where T_{j,o_i}^{start} is the travel time from drone d_j's initial position to object o_i. If s_i follows another task s_k on the same drone, then:

$$t_i^{arr} - t_i^{dep} = T_{j,o_k,o_i}^{obj},$$

where T_{j,o_k,o_i}^{obj} is the travel time for drone d_j from object o_k to object o_i. The finish time is defined as:

$$t_i^{fin} = t_i^{arr} + service_i.$$

The scheduler must also ensure that tasks assigned to the same drone do not overlap. For

two consecutive tasks s_k and s_i assigned to the same drone, this requires:

$$t_k^{fin} \leq t_i^{dep}.$$

The scheduling objective is to minimize the total mission completion time, also known as the makespan:

$$\min \max_i t_i^{fin}.$$

This objective encourages the scheduler to exploit parallelism between drones and avoid unnecessary idle time.

The output of the scheduler is a Python dictionary indexed by drone name. As shown in Figure 3.1, each drone is assigned an ordered list of tasks, where every task contains the target object, required skill, departure time, arrival time, and finish time. This explicit representation makes the generated plan directly inspectable and suitable for execution by the drone processes.

3.4.1 Example

Continuing the same example, the scheduler receives the allocated subtasks together with a precomputed travel-time table:

```
travel_times = {
    "drone_to_object": {
        "Drone1": {"Tower": 4.0, "House2": 6.0},
        "Drone2": {"Tower": 5.0, "House2": 3.0},
        "Drone3": {"Tower": 4.5, "House2": 4.0}
    },
    "drone_object_to_object": {
        "Drone1": {
            "Tower": {"House2": 2.5},
            "House2": {"Tower": 2.5}
        },
        "Drone2": {
            "Tower": {"House2": 2.0},
            "House2": {"Tower": 2.0}
        },
        "Drone3": {
            "Tower": {"House2": 1.5},
            "House2": {"Tower": 1.5}
        }
    }
}
```

One feasible schedule produced by the scheduler is:

```
schedule = {
```

```
    "Drone1": [
        {
            "name": "SubTask1",
            "object": "Tower",
            "skill": "InspectStructure",
            "departure_time": 0.0,
            "arrival_time": 4.0,
            "finish_time": 6.0
        }
    ],
    "Drone2": [
        {
            "name": "SubTask2",
            "object": "House2",
            "skill": "MeasureWind",
            "departure_time": 0.0,
            "arrival_time": 3.0,
            "finish_time": 4.5
        }
    ],
    "Drone3": []
}
```

In this schedule, `SubTask1` is assigned to `Drone1`. Since it is the first task assigned to `Drone1`, the arrival time is computed from the initial drone-to-object travel time:

```
travel_times["drone_to_object"]["Drone1"]["Tower"] = 4.0
```

With an inspection service time of 2.0 seconds, the finish time is $4.0 + 2.0 = 6.0$.

Similarly, `SubTask2` is assigned to `Drone2`. The travel time from `Drone2`'s initial position to `House2` is 3.0 seconds. With a service time of 1.5 seconds, the finish time is $3.0 + 1.5 = 4.5$.

The resulting makespan is therefore:

$$\max(6.0, 4.5) = 6.0.$$

3.5 Replanning

The system supports replanning when the original schedule can no longer be executed as intended. In real drone missions, this may occur due to changes in the drone state, low battery level, poor communication quality, or task requirements that exceed the drone's remaining capabilities. To detect such cases, the system uses an LLM-based task admission module before executing a scheduled task and periodically during flight.

The task admission module receives telemetry information from the drone and information

about the planned task. The telemetry includes the maximum flight time, current battery percentage, battery health, link quality, drone state, estimated flight duration, and estimated task duration. Based on this information, the module classifies the situation into one of three categories: `ok`, `drone_failure`, or `task_failure`. The `ok` decision indicates that the task can be executed as planned. The `drone_failure` decision indicates that the drone is currently not suitable for execution, for example due to poor connection, an emergency state, or insufficient operational readiness. The `task_failure` decision indicates that the task requirements exceed the drone's available resources, for example when the estimated mission duration is longer than the available flight time predicted by the model.

The task admission policy is provided to the LLM through a structured system prompt. The model is instructed to follow the policy and return a JSON object containing both the decision and a short explanation. The output is constrained to the following schema:

```
{
    "decision": "ok | drone_failure | task_failure",
    "reason": "<short explanation>"
}
```

The module evaluates the following conditions:

1. The drone is rejected with `drone_failure` if it is in an invalid state, such as `CONNECTING`, `EMERGENCY`, or `DISCONNECTED`.
2. The drone is rejected with `drone_failure` if the link quality is below the required threshold.
3. The task is rejected with `task_failure` if the required mission time exceeds the available flight time.

The available flight time is estimated from the maximum flight time, battery percentage, and battery health:

$$T_{\text{available}} = T_{\text{max}} \cdot \frac{B_{\text{perc}}}{100} \cdot \frac{B_{\text{health}}}{100}.$$

The required mission time is computed as:

$$T_{\text{required}} = T_{\text{flight}} + T_{\text{task}}.$$

If $T_{\text{available}} < T_{\text{required}}$, the task is rejected with `task_failure`. Otherwise, no rejection rule is triggered and the task is accepted.

When the admission module returns `ok`, the drone proceeds with the scheduled task. When it returns `drone_failure`, the planner removes the affected drone from the set of currently available drones and regenerates the schedule for the unfinished subtasks. When it returns `task_failure`, the planner treats the current assignment as infeasible and attempts to reassign the affected task to another capable drone. In both failure cases, completed tasks are preserved, while unfinished or interrupted tasks are returned to the planning pipeline.

Compared with a purely rule-based replanning module, this LLM-based approach has the advantage that the admission and replanning policies can be modified flexibly through the prompt. For example, new rejection criteria, different battery safety margins, communication thresholds, or task-specific constraints can be added without changing the program logic. This makes the replanning mechanism easier to adapt to different drone platforms, mission types, and safety requirements. At the same time, the use of a structured output schema constrains the LLM response and allows the planner process to parse the decision automatically.

3.6 System Architecture

This chapter describes the architecture of the implemented multi-drone planning and replanning system.

3.6.1 Overview

The architecture is consists of four main parts:

- a planner process responsible for task decompositon, allocation, scheduling, execution monitoring and replanning;
- multiple drone worker processes each representing one drone;
- a world configuration defining objects, skills and drone capabilities;
- a queue based communication system connecting the planner and the drones.

An overview of the implemented architecture is shown in Figure 3.2.

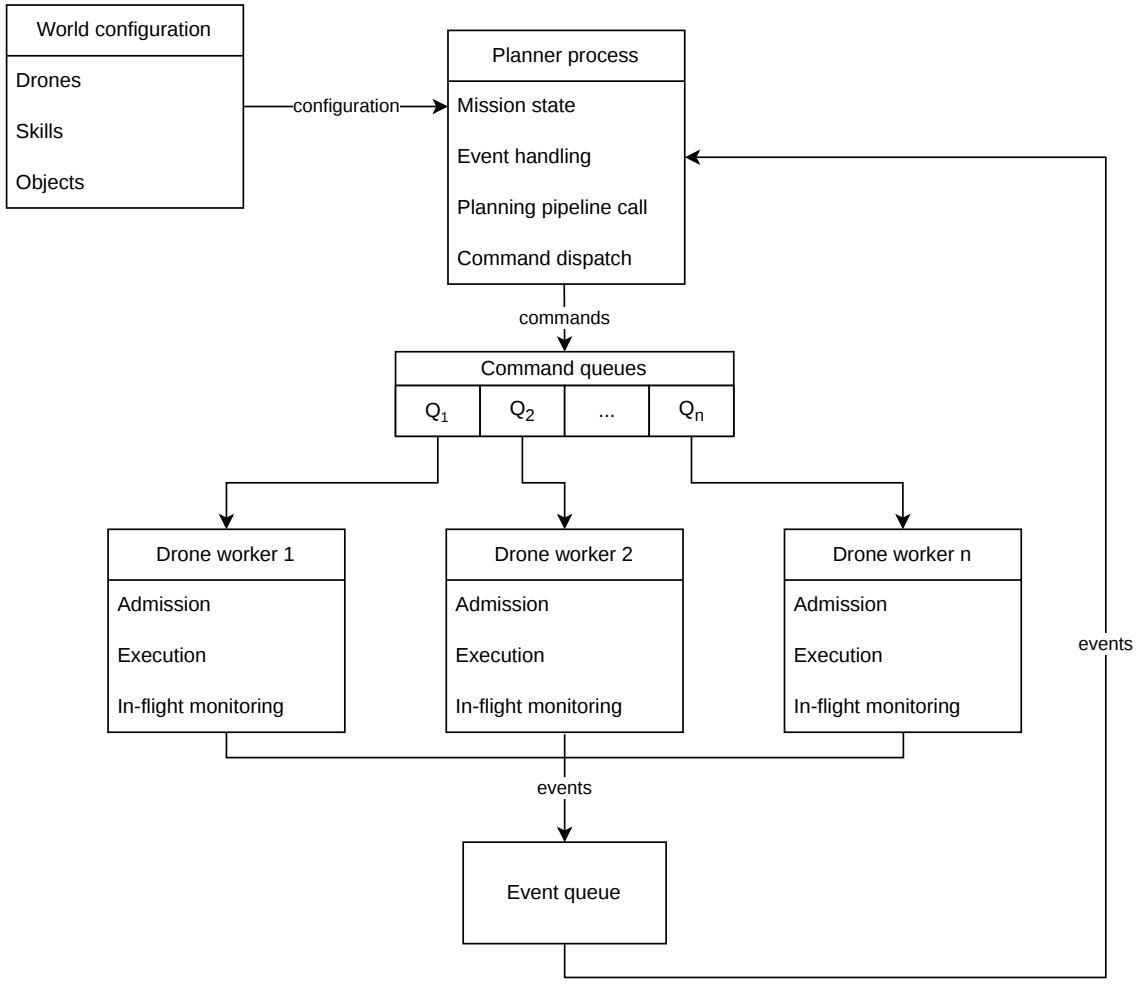


Figure 3.2: Software architecture of the implemented multi-drone planning and replanning system. The planner process receives the world configuration, calls the planning pipeline, dispatches commands through per-drone command queues, and receives execution events through a shared event queue.

3.6.2 System Components

Planner

The planner is the central coordination component of the system. It receives the mission instructions, calls the language models, assigns subtasks to drones and monitors the mission during execution.

The planner maintains the current mission state, including which subtasks are pending, assigned, completed, rejected or failed. Based on this state, it decides whether normal execution should continue or or whether replanning is required. If a drone rejects a task or fails during execution, the planner updates the mission state and attempts to reallocate the remaining work to other available drones.

The planner is also responsible for translating symbolic task assignments into concrete

command messages. These commands are sent to the corresponding drone worker through a dedicated command queue.

Drone Worker Processes

Each drone is represented by a separate worker process. A drone worker receives commands from the planner, checks whether the requested skill can be executed, performs the action, and reports the result back to the planner.

The system supports switching between simulation and real-world modes. In simulation mode, the worker does not communicate with a physical drone. Instead, it is linked with a visualization tool that shows task allocations and drone movement. In real-world mode, the worker sends commands to the actual drone interface.

This process-based design isolates drone execution from the planner. If one drone fails or rejects a task, the planner can handle the event without stopping the entire system.

World Configuration

The world configuration defines the information required by the planner and drone workers. This includes the available drones, with their skills, starting positions, speeds, flight altitudes, and maximum flight times, as well as the positions of objects in the environment.

Communication Architecture

Communication between the planner and drone workers is implemented using multiprocessing queues. The system uses one shared event queue and one command queue per drone.

The command queues are used by the planner to send commands to specific drones. Each drone worker listens only to its own command queue. The shared event queue is used by all drone workers to report execution events back to the planner.

The communication structure can be summarized as follows:

- `command_queues`: a dictionary of per-drone queues used to send commands from the planner to drone workers;
- `event_queue`: a shared queue used by drone workers to send status updates back to the planner;
- drone worker processes: independent processes that execute commands and return events;
- planner process: the central process that dispatches commands and reacts to events.

This architecture supports asynchronous execution. Multiple drones can receive and execute commands independently, while the planner continuously monitors the returned events.

Execution Flow

A mission begins with a natural language instruction provided by the user. The planner sends this instruction, together with information about the available drones, skills, and environment, to the LLM-based planning pipeline. The pipeline returns a schedule containing the generated subtasks, their assigned drones, and their planned execution order.

Each drone is first assigned its initial subtask from the generated schedule. The onboard task admission module then decides whether to accept or reject the proposed subtask. Execution only begins once all drones have accepted their assigned proposals. If any drone rejects its proposed subtask, the schedule is recomputed before execution begins.

Once execution has started, the same onboard module periodically monitors the drone state and reports failures when detected. If no failures are reported, the drones continue according to the original schedule. If a drone fails during execution, the schedule is recomputed for the remaining subtasks that have not yet been completed or started. This replanning step also considers the expected completion times of busy drones, so that drones already executing subtasks can be included in the updated schedule once they become available.

This execution loop continues until all subtasks are completed or no feasible assignment can be found.

Chapter 4

Results

This chapter presents the evaluation results of the proposed LLM-based drone planning and replanning architecture. The evaluation is divided into three parts. First, the individual LLM-based components are evaluated. Second, the selected components are integrated and evaluated in an end-to-end simulation case study. Finally, the system is tested on the physical drone platform to assess its behaviour under real-world execution conditions.

4.1 Component-Level LLM Evaluation

This section evaluates the two LLM-based components used in the proposed architecture: the cloud-based planner and the onboard task admission module. The purpose of this evaluation is to select the models used in the subsequent simulation and physical experiments.

4.1.1 Cloud-based Planning Pipeline

The cloud-based planner was evaluated to determine which model provided the best trade-off between schedule quality, inference time, and reliability. Both cloud-based and locally deployable models were initially considered. However, due to hardware constraints and real-time execution requirements, small quantized LLMs with 1–3 billion parameters were found to be too slow and unreliable for planning, even for tasks consisting of only one or two subtasks. The final cloud-based evaluation therefore used GPT-5, GPT-5-mini, GPT-4o, and GPT-4.1 through the OpenAI API [25].

Evaluation Setup

The models were tested on tasks containing between one and ten subtasks. For each subtask count, three task descriptions were evaluated. The task descriptions were written in unstructured natural language with varied wording and sentence structure in order to test the models' ability to interpret high-level mission instructions.

Table 4.1: Average makespan difference from the VRP baseline by number of subtasks. Values in parentheses indicate solved cases over three tasks.

model Subtasks	GPT-5	GPT-5-mini	GPT-4o	GPT-4.1
1	0.00 (3/3)	0.00 (3/3)	0.00 (3/3)	0.00 (3/3)
2	0.00 (3/3)	0.00 (3/3)	14.07 (3/3)	0.00 (3/3)
3	0.00 (3/3)	0.00 (3/3)	36.54 (3/3)	62.60 (3/3)
4	0.00 (3/3)	0.00 (3/3)	121.29 (3/3)	121.29 (3/3)
5	0.00 (3/3)	0.00 (3/3)	132.79 (3/3)	112.17 (3/3)
6	0.00 (3/3)	0.86 (3/3)	78.51 (3/3)	– (0/3)
7	0.00 (3/3)	5.77 (2/3)	62.95 (3/3)	52.03 (2/3)
8	0.00 (3/3)	14.90 (3/3)	– (0/3)	– (0/3)
9	0.56 (3/3)	15.88 (2/3)	22.16 (2/3)	79.66 (1/3)
10	4.35 (3/3)	25.19 (3/3)	– (0/3)	161.43 (1/3)

The generated schedules were compared against an optimal schedule computed using a VRP solver [26]. The VRP solution was used as a baseline for evaluating the makespan of the LLM-generated mission schedules. A generated schedule was considered successful if it produced a valid sequence of drone actions satisfying all requested subtasks.

Schedule Quality

Table 4.1 shows the average gap between the makespan generated by each model and the VRP baseline. The table also reports the number of successfully generated schedules out of three for each subtask count.

Inference Time

Table 4.2 shows the average inference time for each model, computed only over successfully generated schedules.

Model Selection

The results show that GPT-5 consistently generated schedules close to the VRP baseline, but required substantially longer inference times than GPT-5-mini. GPT-5-mini produced valid schedules for most tasks and achieved near-optimal schedules for tasks with up to five subtasks, while requiring considerably less inference time. GPT-4o and GPT-4.1 performed well on simpler tasks, but increasingly produced suboptimal or invalid schedules as the number of subtasks increased.

Based on this trade-off between schedule quality, inference time, and API cost, GPT-5-mini was selected for the cloud-based planning pipeline.

Table 4.2: Average inference time by number of subtasks. Values in parentheses indicate solved cases over three tasks.

model Subtasks	GPT-5	GPT-5-mini	GPT-4o	GPT-4.1
1	32.97 (3/3)	20.27 (3/3)	6.17 (3/3)	4.07 (3/3)
2	49.90 (3/3)	21.87 (3/3)	5.00 (3/3)	4.03 (3/3)
3	94.50 (3/3)	33.50 (3/3)	8.93 (3/3)	7.13 (3/3)
4	111.90 (3/3)	51.63 (3/3)	12.90 (3/3)	7.13 (3/3)
5	112.33 (3/3)	45.83 (3/3)	15.63 (3/3)	11.30 (3/3)
6	205.07 (3/3)	98.10 (3/3)	14.47 (3/3)	– (0/3)
7	284.37 (3/3)	119.30 (2/3)	17.90 (3/3)	17.00 (2/3)
8	284.77 (3/3)	133.67 (3/3)	– (0/3)	– (0/3)
9	243.07 (3/3)	112.05 (2/3)	19.70 (2/3)	16.50 (1/3)
10	269.00 (3/3)	97.73 (3/3)	– (0/3)	16.80 (1/3)

4.1.2 Onboard Task Admission

The onboard task admission module has different requirements from the cloud-based planner. While the planner must produce high-quality mission schedules, the admission module must evaluate drone telemetry quickly and reliably during execution. Since this module directly affects whether a task is accepted, rejected, or sent for replanning, reliability is particularly important.

Evaluation Setup

The evaluated models were tested on four categories: acceptable tasks, drone-state errors, link-quality errors, and flight-time errors. Each category contained 20 test cases, resulting in 80 test cases per model. Seven 4-bit quantized models, ranging from 0.6 to 4 billion parameters, were evaluated from the Qwen [27, 28], Gemma [29], Llama [30], and Phi [31] model families.

Classification Accuracy

Table 4.3 reports the number of correctly classified cases out of 20 for each category.

Inference Time

Table 4.4 reports the average and maximum inference time for each model.

Table 4.3: Task admission accuracy across failure categories. Each value indicates the number of correctly solved cases out of 20.

Model	Acceptable task	Drone state error	Link quality error	Flight time error
qwen3:0.6b	18	8	6	16
gemma3:1b	20	15	0	0
qwen3:1.7b	20	20	20	20
qwen2.5:3b	7	20	20	9
llama3.2:3b	13	20	18	0
phi4-mini:3.8b	20	20	20	1
gemma3:4b	14	20	9	0

Table 4.4: Average and maximum inference time for the task admission module.

Model	Average inference time (s)	Maximum inference time (s)
qwen3:0.6b	7.99	24.75
gemma3:1b	0.76	2.83
qwen3:1.7b	3.60	7.45
qwen2.5:3b	1.02	6.44
llama3.2:3b	1.25	14.24
phi4-mini:3.8b	1.52	7.72
gemma3:4b	1.20	9.12

Model Selection

The results show that qwen3:1.7b achieved perfect classification accuracy across all four categories, correctly solving all 80 test cases. Although some models achieved lower average inference times, they failed on safety-relevant categories such as link-quality or flight-time errors. Since the task admission module directly affects replanning decisions, classification reliability was prioritized over minimal inference time. Therefore, qwen3:1.7b was selected for onboard task admission.

4.2 Simulation Results

This section evaluates the planning and task admission components in a simulated multi-drone mission environment. The purpose of the simulation is to validate the interaction between cloud-based planning, onboard task admission, execution monitoring, failure detection, and replanning before testing the system on the physical drone platform.

4.2.1 Simulation Setup

The simulation environment was designed to represent a multi-drone inspection scenario with static objects of interest distributed across a two-dimensional area.

The simulated drone team consists of multiple drones with predefined capabilities. Depending on the assigned subtask, a drone may be required to capture RGB images, record video, acquire thermal images, inspect structures, or measure wind levels. The simulator abstracts away low-level flight dynamics and instead focuses on task-level execution, including task allocation, admission checking, failure handling, and replanning.

To evaluate the system response to execution-time changes, the simulator also models battery drainage, link-quality variations, and drone failures. Battery levels decrease during mission execution, while link quality may change over time and affect whether a drone can safely accept a task. Drone failures can occur during execution, causing the affected drone's remaining subtasks to be returned to the cloud-based planner for rescheduling. These simulated disturbances allow the replanning and task admission mechanisms to be evaluated under dynamic mission conditions.

4.2.2 Simulation Case Study

A representative mission execution was conducted in the simulation environment using the selected language models for planning and task admission. The drone team executed the following mission:

> Document the condition of all houses with video and inspect each rooftop, while measuring wind levels near the Base and Tower. In addition, take an RGB image of the Tower.

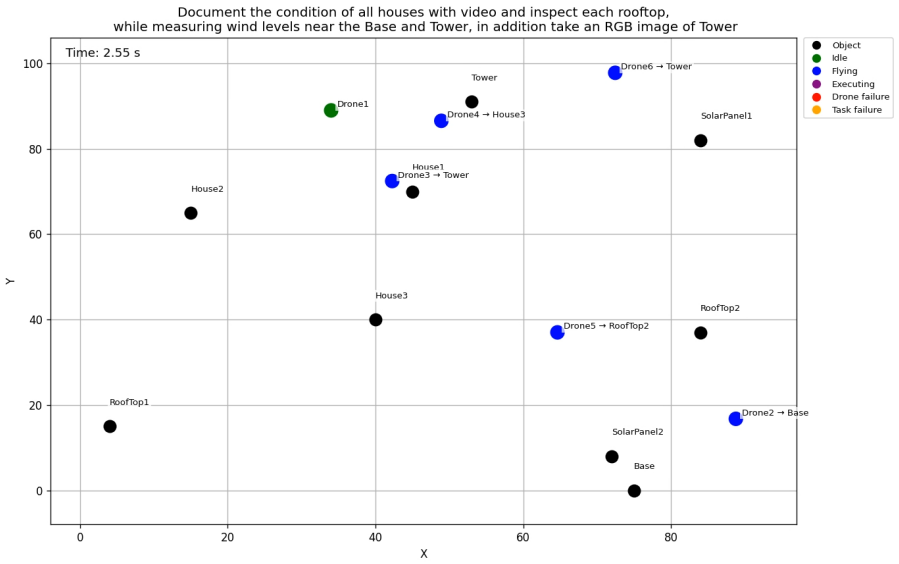
Based on the submitted natural-language mission, the cloud-based planner generated a schedule, and the resulting subtasks were sent to the drones for onboard admission checking.

Figures 4.1 to 4.3 show the main stages of the execution:

1. All drones accept their assigned subtasks and start moving toward their targets (Figure 4.1a).
2. Once the drones arrive at their assigned targets, they begin executing their subtasks (Figure 4.1b).
3. Drone5's onboard admission module signals failure during the execution phase of SubTask4. The affected remaining subtasks are sent back to the cloud-based planner for rescheduling (Figure 4.2a).
4. While the updated schedule is being generated, the other drones continue executing their previously allocated subtasks (Figure 4.2b).

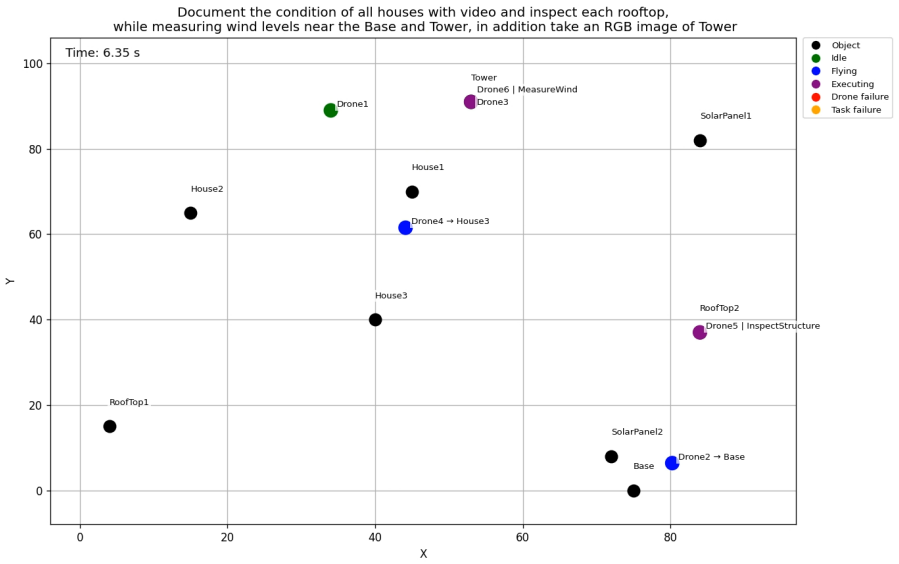
5. SubTask4 is reassigned to Drone6 and acknowledged by the onboard admission module (Figure 4.3a).
6. All subtasks are completed by the drone team (Figure 4.3b).

This qualitative case study demonstrates that the implemented architecture can coordinate the cloud-based planner, onboard task admission module, and execution monitor within a single mission workflow. In particular, the sequence shows that the system can detect a drone failure, update the mission state, reschedule affected subtasks, and continue execution with the remaining available drones.




```
PENDING SUBTASKS
----------------------------
SubTask1: House1
SubTask2: House2
SubTask3: House3
   ✓ Drone4 ACK
SubTask4: RoofTop1
SubTask5: RoofTop2
   ✓ Drone5 ACK
SubTask6: Base
   ✓ Drone2 ACK
SubTask7: Tower
   ✓ Drone6 ACK
SubTask8: Tower
   ✓ Drone3 ACK
```

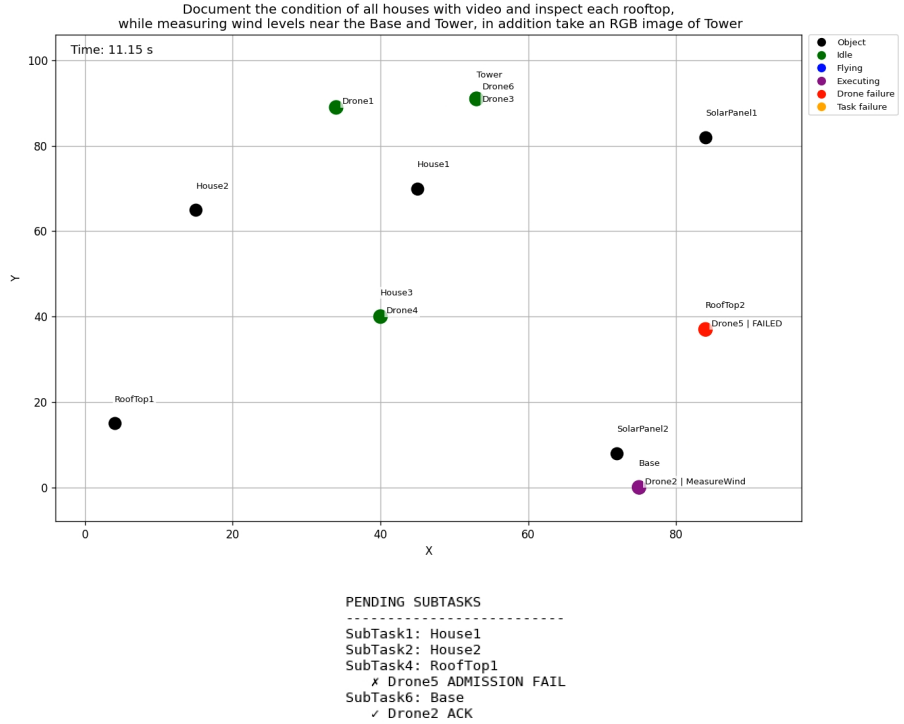
(a) First allocation round.



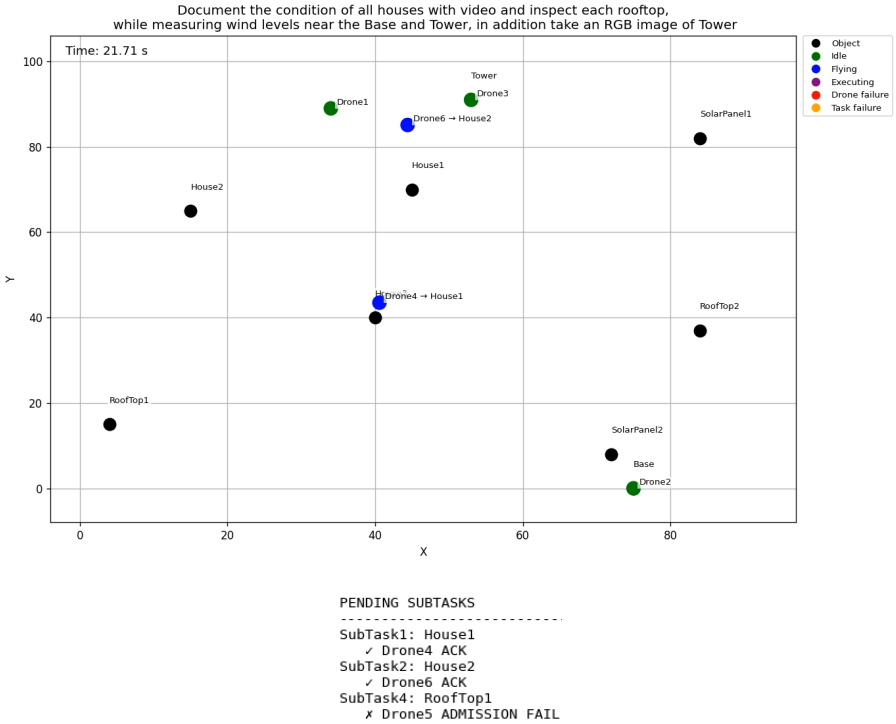

```
PENDING SUBTASKS
----------------------------
SubTask1: House1
SubTask2: House2
SubTask3: House3
   ✓ Drone4 ACK
SubTask4: RoofTop1
SubTask5: RoofTop2
   ✓ Drone5 ACK
SubTask6: Base
   ✓ Drone2 ACK
SubTask7: Tower
   ✓ Drone6 ACK
```

(b) The arrived drones start executing their tasks.

Figure 4.1: Initial stages of the simulated mission execution.

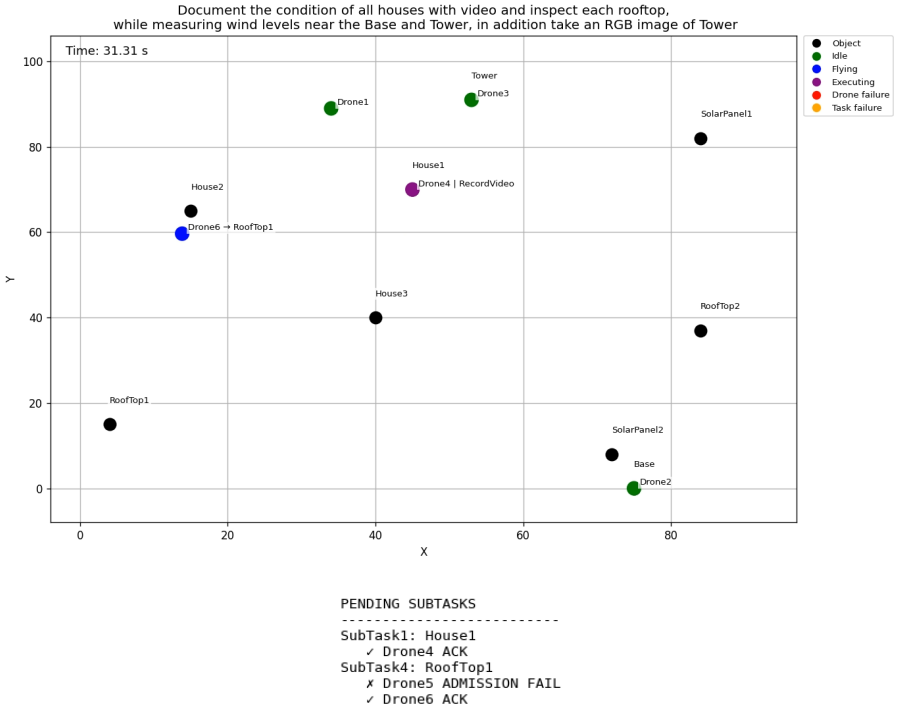


(a) Drone5's onboard admission module signals failure during task execution.

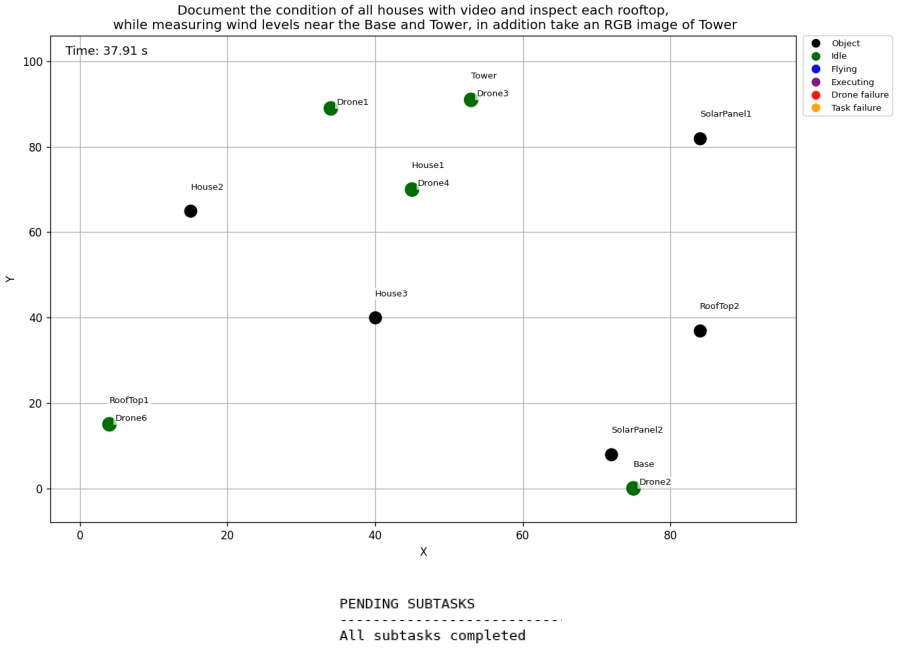


(b) The remaining drones continue executing their originally allocated subtasks while the cloud-based planner reschedules the failed task.

Figure 4.2: Intermediate stages of the simulated mission execution.



(a) SubTask4 is allocated and acknowledged by Drone6.



(b) All subtasks are completed.

Figure 4.3: Final stages of the simulated mission execution.

4.3 Physical Platform and Experimental Setup

This section describes the physical drone platform, laboratory environment, and control interface used as the basis for real-world deployment of the proposed architecture.

4.3.1 Scope of Physical Integration

The full closed-loop architecture, including cloud-based planning, onboard LLM-based task admission, failure detection, and replanning, was not deployed on the physical drones within the scope of this thesis. Instead, the physical platform is described to show the intended deployment environment and the connection between the symbolic drone skills used by the planner and the capabilities of the Anafi drones.

The complete system behavior is therefore evaluated in the simulation results, while the physical platform description provides context for future real-world deployment.

4.3.2 Drone Platform

The physical platform was based on drones from the Parrot Anafi family. To represent a drone team with heterogeneous sensing capabilities, four variants were considered: the Parrot Anafi 4K, Anafi Thermal, Anafi USA, and Anafi Ai. These platforms are shown in Figure 4.4.

(a) Anafi 4K. **(b)** Anafi Thermal.

(c) Anafi USA. **(d)** Anafi Ai.

Figure 4.4: Drone platforms used during physical testing. Images adapted from Parrot product documentation [32–35].

Table 4.5 summarizes the project-relevant specifications of the drone variants used in the

experiments, together with the symbolic skills assigned to each platform in the planner.

Table 4.5: Capabilities of the Anafi drone variants used in the project. Specifications adapted from [36].

Capability	Anafi 4K	Anafi Thermal	Anafi USA	Anafi Ai
Max. flight time [min]	25	26	32	32
Max. horizontal speed [m/s]	15	15	14.7	16
RGB camera [MP]	21	21	21	48
Video capability	4K/FHD	4K/FHD/HD	4K/FHD/HD	4K/FHD
Thermal sensing	No	Yes	Yes	No
Zoom [x]	1–3	1–3	1–32	1–6
Assigned skills	RGB image Video Zoom image	RGB image Thermal image Zoom image Video Dual-spectral	RGB image Thermal image Zoom image Video Dual-spectral	RGB image Video Zoom image

Note: Dual-spectral refers to combined RGB and thermal inspection.

4.3.3 Laboratory and Control Setup

The intended physical setup consists of an indoor laboratory environment, a Vicon-based localization system, and a ROS-based control interface for communicating with the Anafi drones. This setup provides the infrastructure required for executing high-level mission commands on the physical drone platform.

Laboratory Environment

The laboratory environment is equipped with a Vicon motion capture system [37]. Since GPS-based localization is not available indoors, the Vicon system can provide position and orientation feedback during flight. Reflective markers can be mounted on the drone, allowing the motion capture system to track the drone within the available flight area.

Similar target objects to those used in the simulation environment can be defined in the physical setup. Unlike the two-dimensional simulation setup, the laboratory environment is represented in three dimensions. Each target object can be assigned a position in the Vicon coordinate frame, allowing the drone to navigate to target locations using 3D position feedback.

Drone Control Interface

The physical setup uses the `anafi_autonomy` ROS package [36] as the control interface between the mission execution pipeline and the Parrot Anafi drones. The package provides ROS topics and services for controlling the drone and accessing sensor data. In the proposed

integration, it would be used to execute high-level mission commands such as takeoff, landing, and navigation.

The LLM-based planner does not directly control the drone hardware. Instead, it generates symbolic subtasks, which are translated by the mission execution layer into commands supported by the drone control interface. This separation allows the planning system to remain platform-independent, while the execution layer handles platform-specific communication with the Anafi drones.

Chapter 5

Conclusion and Future Work

Bibliography

[1] R. H. Jacobsen, L. Matlekovic, L. Shi, N. Malle, N. Ayoub, K. Hageman, S. Hansen, F. F. Nyboe, and E. Ebeid, "Design of an autonomous cooperative drone swarm for inspections of safety critical infrastructure," *Applied Sciences*, vol. 13, no. 3, p. 1256, 2023.

[2] M. A. Dinesh, S. S. Kumar, S. J. Shetty, A. K. N., and M. K. M. Gowda, "Development of an autonomous drone for surveillance application," *International Research Journal of Engineering and Technology (IRJET)*, vol. 5, no. 8, p. 331, September 2018.

[3] K. Anderson and K. J. Gaston, "Lightweight unmanned aerial vehicles will revolutionize spatial ecology," *Frontiers in Ecology and the Environment*, vol. 11, no. 3, pp. 138–146, 2013.

[4] A. Ellenberg, A. Kontsos, F. Moon, and I. Bartoli, "Bridge deck delamination identification from unmanned aerial vehicle infrared imagery," *Automation in Construction*, vol. 72, pp. 155–165, 2016.

[5] M. Erdelj and E. Natalizio, "Uav-assisted disaster management: Applications and open issues," in *International Conference on Computing, Networking and Communications (ICNC)*. Kauai, HI, USA: IEEE, Feb 2016, pp. 1–5, hal-01305371.

[6] K. Zhou, Z. Wang, Y.-Q. Ni, Y. Zhang, and J. Tang, "Unmanned aerial vehicle-based computer vision for structural vibration measurement and condition assessment: A concise survey," *Journal of Infrastructure Intelligence and Resilience*, vol. 2, no. 2, p. 100031, 2023.

[7] Amazon, "Prime air," https://www.amazon.com/primeair, n.d., accessed: 2026-04-25.

[8] Invest in Denmark, "Denmark launches first long-range drone delivery service," https://investindk.com/insights/health-drone, May 2022, accessed: 2026-04-25.

[9] P. Nooralishahi, C. Ibarra-Castanedo, S. Deane, F. López, S. Pant, M. Genest, N. P. Avdelidis, and X. P. V. Maldague, "Drone-based non-destructive inspection of industrial sites: A review and case studies," *Drones*, vol. 5, no. 4, p. 106, 2021.

[10] H. M. Rostum and J. Vásárhelyi, "A review of using visual odometry methods in autonomous uav navigation in gps-denied environment," *Acta Universitatis Sapientiae, Electrical and Mechanical Engineering*, vol. 15, pp. 14–32, 2023.

[11] L. Wallace, A. Lucieer, Z. Malenovský, D. Turner, and P. Vopěnka, "Assessment of forest structure using two uav techniques: A comparison of airborne laser scanning and structure from motion (sfm) point clouds," *Forests*, vol. 7, no. 3, p. 62, 2016.

[12] P. McEnroe, S. Wang, and M. Liyanage, "A survey on the convergence of edge computing and ai for uavs: Opportunities and challenges," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15 435–15 459, 2022.

[13] M. Wooldridge, *An Introduction to MultiAgent Systems*. Chichester, West Sussex, England: John Wiley & Sons Ltd, 2002, reprinted August 2002.

[14] G. Pantelimon, K. Tepe, R. Carriveau, and S. Ahmed, "Survey of multi-agent communication strategies for information exchange and mission control of drone deployments," *Journal of Intelligent & Robotic Systems*, vol. 95, no. 3-4, pp. 779–788, 2019.

[15] E. H. Durfee, "Distributed problem solving and planning," in *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*, G. Weiss, Ed. Cambridge, Massachusetts: MIT Press, 1999, pp. 121–164.

[16] T. Bektas, "The multiple traveling salesman problem: An overview of formulations and solution procedures," *Omega*, vol. 34, no. 3, pp. 209–219, 2006.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[18] Y. Kim, D. Kim, J. Choi, J. Park, N. Oh, and D. Park, "A survey on integration of large language models with intelligent robots," *Intelligent Service Robotics*, vol. 17, pp. 1091–1107, 2024.

[19] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022. [Online]. Available: https://arxiv.org/abs/2204.01691

[20] W. Wang, Y. Li, L. Jiao, and J. Yuan, "Gsce: A prompt framework with enhanced reasoning for reliable llm-driven drone control," *arXiv preprint arXiv:2502.12531*, 2025. [Online]. Available: https://arxiv.org/abs/2502.12531

[21] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2307.04738

[22] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "Smart-llm: Smart multi-agent robot task planning using large language models," *arXiv preprint arXiv:2309.10062*, 2024. [Online]. Available: https://arxiv.org/abs/2309.10062

[23] S. Borate, B. Rai B, V. Pardeshi, and M. Vadali, "Llm-based generalizable hierarchical task planning and execution for heterogeneous robot teams with event-driven replanning," *arXiv preprint arXiv:2511.22354*, 2025. [Online]. Available: https://arxiv.org/abs/2511.22354

[24] T. Yang, P. Feng, Q. Guo, J. Zhang, X. Zhang, J. Ning, X. Wang, and Z. Mao, "Autohma-llm: Efficient task coordination and execution in heterogeneous multi-agent systems using hybrid large language models," *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 2, pp. 987–998, 2025.

[25] OpenAI, "Models," https://developers.openai.com/api/docs/models, 2026, accessed: 2026-05-01.

[26] Google Developers, "Vehicle routing problem (vrp)," 2026, accessed: 2026-05-01. [Online]. Available: https://developers.google.com/optimization/routing/vrp

[27] Ollama, "Qwen2.5," https://ollama.com/library/qwen2.5, accessed: 2026-05-01.

[28] ——, "Qwen3," https://ollama.com/library/qwen3, accessed: 2026-05-01.

[29] ——, "Gemma 3," https://ollama.com/library/gemma3, accessed: 2026-05-01.

[30] ——, "Llama 3.2," https://ollama.com/library/llama3.2, accessed: 2026-05-01.

[31] ——, "Phi-4," https://ollama.com/library/phi4, accessed: 2026-05-01.

[32] Parrot, "Anafi product sheet / white paper," 2021, pDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2021-02/anafi-product-sheet-white-paper-en.pdf

[33] ——, "Anafi thermal user guide," 2021, user guide, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2021-09/anafi-thermal-user-guide.pdf

[34] ——, "Anafi usa product sheet," 2023, pDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2023-02/ANAFI-USA-product-sheet.pdf

[35] ——, "Anafi ai product sheet," 2023, pDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2023-02/ANAFI-Ai-product-sheet.pdf

[36] A. Sarabakha, "anafi_ros: From off-the-shelf drones to research platforms," *arXiv preprint arXiv:2303.01813*, 2023. [Online]. Available: https://arxiv.org/abs/2303.01813

[37] Vicon Motion Systems, "Tracker Motion Tracking Software," https://www.vicon.com/software/tracker/, accessed: 2026-05-14.